Module 08 Lab - The Bias-Variance Tradeoff

Objective: To understand and visualize the concepts of **overfitting**, **underfitting**, and the **bias-variance tradeoff**, which are central to building models that generalize well to new data.

In this lab, you will train models of varying complexity and plot their performance to see these concepts in action.

Part 1: Understanding the Concepts

- **Underfitting (High Bias):** The model is **too simple** and fails to capture the underlying patterns in the data. It performs poorly on both the training data and the test data. It has high bias because it makes strong, incorrect assumptions about the data.
- **Overfitting (High Variance):** The model is **too complex** and learns the training data too well, including the noise and random fluctuations. It performs exceptionally well on the training data but poorly on the test data because it has memorized the training set instead of learning the general pattern. It has high variance because its performance changes drastically with different training data.
- **The Goal:** Find a model that is "just right"—complex enough to capture the true pattern but not so complex that it memorizes the noise. This is the **bias-variance tradeoff**.

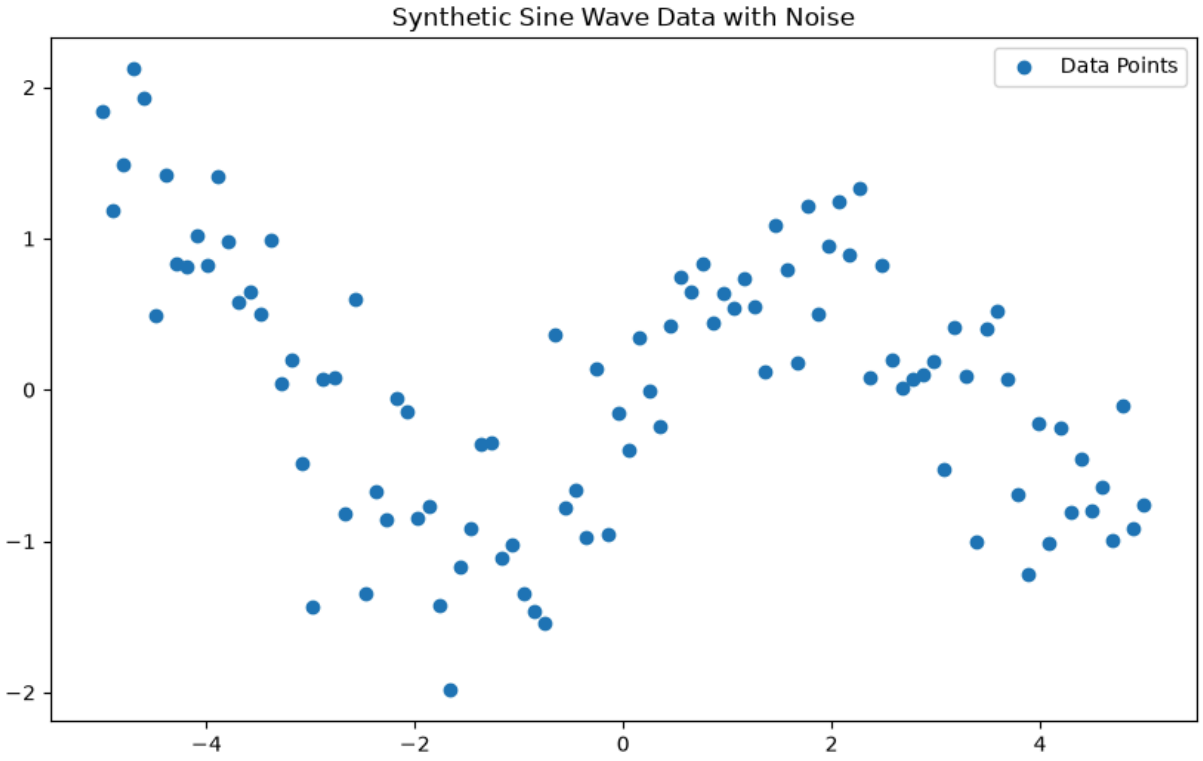
Part 2: Setup

To visualize this, we will create a synthetic (fake) dataset with a known pattern and some noise.

In [1]:
```
import numpy as np  # Import NumPy for numerical operations and array handli
import matplotlib.pyplot as plt  # Import Matplotlib's plotting interface.
from sklearn.pipeline import make_pipeline  # Import a helper for chaining m
from sklearn.preprocessing import PolynomialFeatures  # Import a transformer
from sklearn.linear_model import LinearRegression  # Import the linear regre
from sklearn.model_selection import learning_curve  # Import a tool for eval

# Generate some synthetic data
np.random.seed(0)  # Set the random seed so the noise is reproducible.
X = np.linspace(-5, 5, 100)  # Create 100 evenly spaced input values from -5
y = np.sin(X) + np.random.normal(0, 0.5, 100)  # Create sine-wave outputs wi
X = X[:, np.newaxis]  # Reshape X into a two-dimensional array for scikit-le
```

```
# Plot the data to see the underlying sine wave pattern
plt.figure(figsize=(10, 6))  # Create a figure that is 10 inches wide and 6
plt.scatter(X, y, label='Data Points')  # Plot the input and output values a
plt.title('Synthetic Sine Wave Data with Noise')  # Set the plot's title.
plt.legend()  # Display the legend for the plotted data.
plt.show()  # Display the plot.
```



Part 3: Modeling with Different Complexities

We will use **Polynomial Regression** to control model complexity. A polynomial of degree 1 is a simple straight line (underfitting). A polynomial of degree 15 is a very complex, wiggly line (overfitting).

Your Task: Train and visualize three models with different degrees (1, 4, and 15) to see underfitting, a good fit, and overfitting.

Task 1: Train and Plot Models

Your Task: For each degree (1, 4, 15), create a polynomial regression model, train it, and plot its predictions against the original data.

In [2]:
```
# --- ENTER YOUR CODE HERE ---

plt.figure(figsize=(14, 8))  # Create a figure that is 14 inches wide and 8
plt.scatter(X, y, label='Data Points')  # Plot the data points.

degrees = [1, 4, 15]  # Set the polynomial degrees to compare.
```

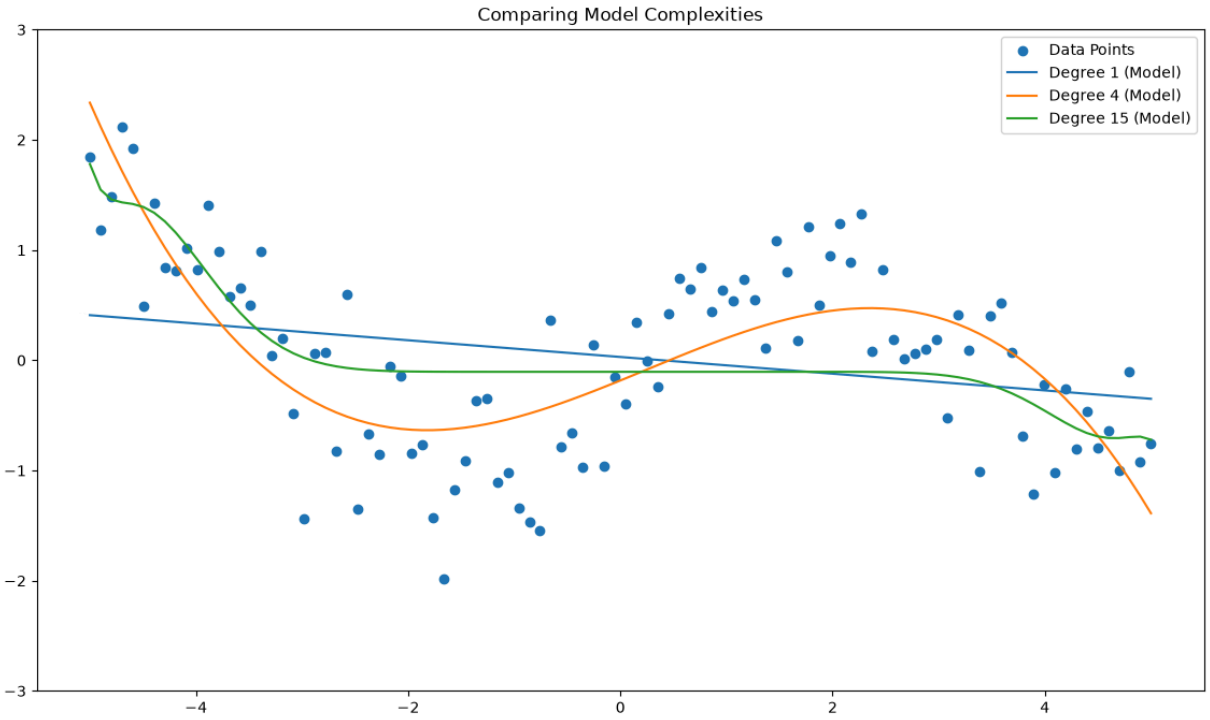
```
for degree in degrees:  # Repeat the following steps for each degree.
    # 1. Create a pipeline that adds polynomial features and then fits a lin
    model = make_pipeline(PolynomialFeatures(degree), LinearRegression())  #

    # 2. Train the model
    model.fit(X, y)  # Fit the model to the input data and target values.

    # 3. Make predictions
    y_pred = model.predict(X)  # Predict target values for the input data.

    # 4. Plot the model's predictions
    plt.plot(X, y_pred, label=f'Degree {degree} (Model)')  # Plot the predic

plt.legend()  # Display the legend.
plt.title('Comparing Model Complexities')  # Set the plot title.
plt.ylim(-3, 3)  # Set the vertical axis limits.
plt.show()  # Display the plot.
```



Part 4: Learning Curves

Concept: A **learning curve** is a powerful tool to diagnose bias and variance. It plots the model's performance (e.g., accuracy or error) on both the **training set** and the **validation set** as a function of the number of training samples.

- **High Bias (Underfitting):** Both the training score and validation score will be low and will plateau quickly. The model is too simple to learn from more data.
- **High Variance (Overfitting):** There will be a large gap between the high training score and the low validation score. The model memorized the training data but can't generalize.
- **Just Right:** The training and validation scores will converge to a high value.

Task 2: Plot Learning Curves

Your Task: Use the `learning_curve` function from scikit-learn to plot the learning curves for the underfit (degree 1) and overfit (degree 15) models.

In [3]:
```
def plot_learning_curve(estimator, title, X, y, ylim=None, cv=None, n_jobs=N
    plt.figure(figsize=(10,6))  # Create the figure for the plot.
    plt.title(title)  # Set the plot title.
    if ylim is not None:  # Check whether custom y-axis limits were provided
        plt.ylim(*ylim)  # Apply the provided y-axis limits.
    plt.xlabel("Training examples")  # Label the x-axis.
    plt.ylabel("Score")  # Label the y-axis.
    train_sizes, train_scores, test_scores = learning_curve(  # Calculate sc
        estimator, X, y, cv=cv, n_jobs=n_jobs, train_sizes=train_sizes)  # F
    train_scores_mean = np.mean(train_scores, axis=1)  # Calculate the mean
    train_scores_std = np.std(train_scores, axis=1)  # Calculate the standar
    test_scores_mean = np.mean(test_scores, axis=1)  # Calculate the mean cr
    test_scores_std = np.std(test_scores, axis=1)  # Calculate the standard
    plt.grid()  # Display grid lines on the plot.

    plt.fill_between(train_sizes, train_scores_mean - train_scores_std,  # S
                     train_scores_mean + train_scores_std, alpha=0.1,  # Set
                     color="r")  # Color the training-score area red.
    plt.fill_between(train_sizes, test_scores_mean - test_scores_std,  # Sha
                     test_scores_mean + test_scores_std, alpha=0.1, color="g
    plt.plot(train_sizes, train_scores_mean, 'o-', color="r",  # Plot the me
             label="Training score")  # Label the training-score line.
    plt.plot(train_sizes, test_scores_mean, 'o-', color="g",  # Plot the mea
             label="Cross-validation score")  # Label the cross-validation-s

    plt.legend(loc="best")  # Display the legend in the best location.
    return plt  # Return the plotting interface.

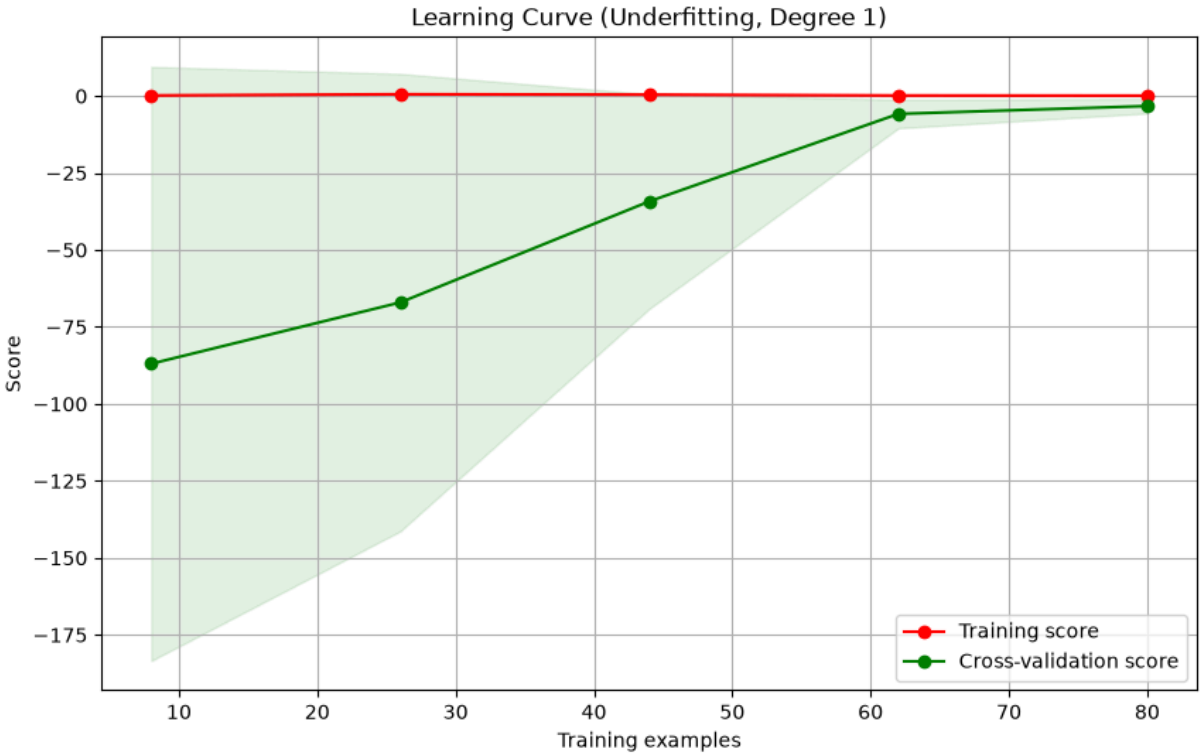
# --- ENTER YOUR CODE HERE ---

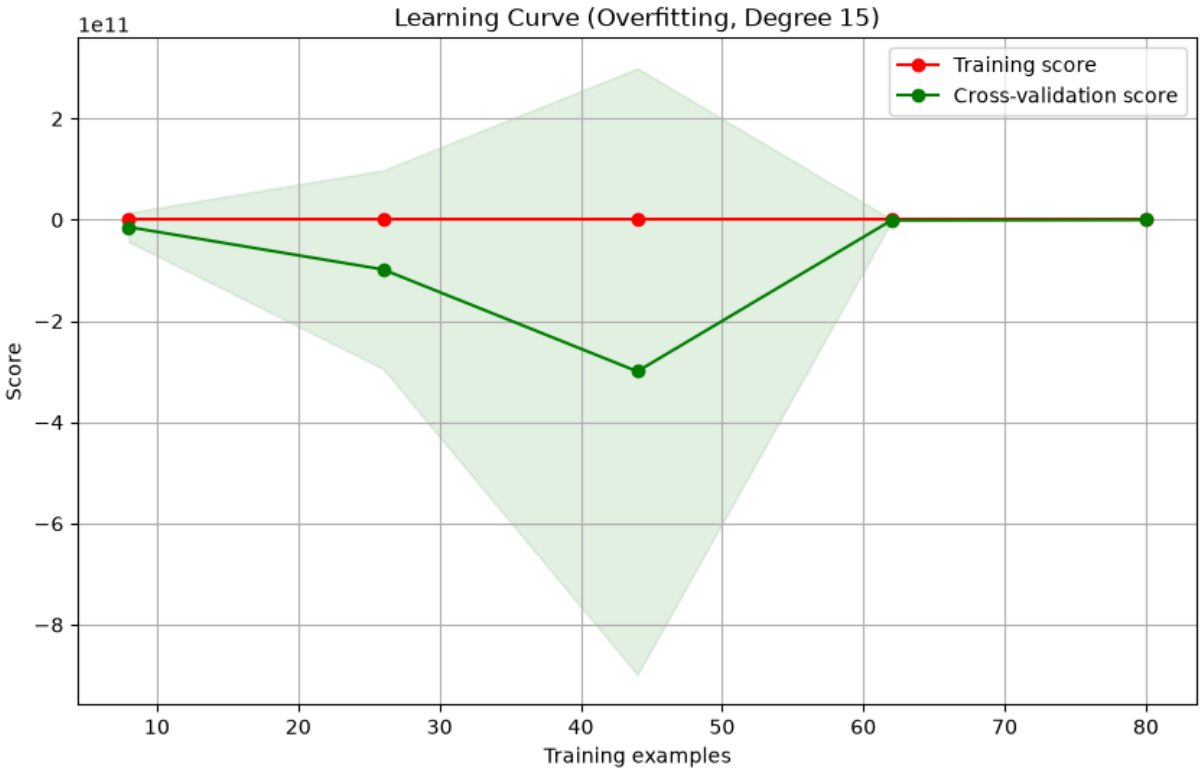
# 1. Create the underfit and overfit models
underfit_model = make_pipeline(PolynomialFeatures(1), LinearRegression())  #
```

```
overfit_model = make_pipeline(PolynomialFeatures(15), LinearRegression())  #

# 2. Plot the learning curve for the underfit model
plot_learning_curve(underfit_model, "Learning Curve (Underfitting, Degree 1)
plt.show()  # Display the underfit model's learning curve.

# 3. Plot the learning curve for the overfit model
plot_learning_curve(overfit_model, "Learning Curve (Overfitting, Degree 15)"
plt.show()  # Display the overfit model's learning curve.
```





📝 Knowledge Check

Instructions: Answer the following questions in this markdown cell.

1. **In the first plot of the three models, which model (degree 1, 4, or 15) is underfitting, which is overfitting, and which is a good fit? Explain your reasoning.**
2. **Looking at the learning curve for the underfitting model, what do you observe about the training and cross-validation scores?** What does this tell you?
3. **Looking at the learning curve for the overfitting model, what do you observe about the gap between the training and cross-validation scores?** What does this tell you?

[ENTER YOUR ANSWERS HERE]

1. Degree of 1 is an example of underfitting as the model is too simple to accurately match the data. Degree of 15 is an example of overfitting as the curve is too complicated. Degree of 4 is a good model as it matches the pattern.

2. Underfitting will produce low training and cross validation scores as the model is too simple to pick up patterns within the data.

3. Overfitting will produce a large difference between the training and cross validation score. This indicates that the model does not generalise well as it performs well on training data but cannot predict accurately on new data.